Reproducibility Practices in Statistical Research Software: A Case Study of JASA Publications

Mrudula Reddy Atla
University of California, Santa Cruz
matla@ucsc.edu

Abstract

Reproducibility is a cornerstone of scientific credibility, yet challenges in replicating computational research persist across disciplines. This paper presents a systematic evaluation of reproducibility in Statistical Research Software Engineering (SRSE) by assessing a curated set of computational research papers from the Journal of the American Statistical Association (JASA), published between 2020 and 2025. We propose a five-indicator reproducibility assessment rubric encompassing Data Availability, Code Accessibility, Code Executability, Documentation Completeness, and Result Reproducibility, each weighted to reflect its significance in the reproduction pipeline. A total of 20 papers were selected through a randomized sampling approach, of which 16 had available software artifacts, and 10 achieved non-zero reproducibility scores. Our findings reveal a significant gap between artifact availability and actual reproducibility, primarily due to incomplete documentation, missing environment specifications, and code execution failures. We highlight the need to complement compliance "sticks" with better "carrots" ie. recognition, credit, and incentives tied to executable artifacts and validated reproducibility, to motivate sustained adoption of good practices in SRSE [45].

1 Introduction

Scientific reproducibility is an essential pillar of modern research, enabling the verification of results, fostering trust in methodologies, and allowing future researchers to build upon prior work. However, the meaning and standards of reproducibility vary not only across disciplines but even within the same field. The widely recognized "reproducibility crisis" (Baker, 2016) has highlighted the pressing need for structured and systematic approaches to ensure research can be reliably reproduced. A survey of 1,576 researchers published in Nature indicated that over 70% of researchers failed to reproduce another researcher's experiments, and more than half could not reproduce their own (Baker, 2016). These challenges span issues from code execution to inconsistencies in experimental documentation, necessitating robust frameworks to ensure transparent and reliable scientific practices (Desai, Abdelhamid, & Padalkar, 2025).

In this paper, we use **Statistical Research Software (SRS)** to mean software, scripts, and computational workflows developed to conduct *statistical* analyses data processing, modeling, inference, simulation, visualization within research articles. Correspondingly, we use **Statistical Research Software Engineering (SRSE)** to refer to the application of software engineering principles (versioning, testing, packaging, environment specification, documentation) to such statistical research software. All uses of "SRSE" in this study are intended in this specific statistical context (i.e., SRSE), and our evaluation focuses on artifacts accompanying JASA papers from 2020–2025.

Statistical Research Software Engineering (SRSE) is an emerging discipline that applies software engineering principles to develop and maintain research software, with the goal of ensuring that it is robust, efficient, and reusable, thereby supporting reproducible science. In this study, we define **Computational Reproducibility** in the context of SRSE as the condition under which (i) the original data are publicly available, (ii) the code used for analysis is accessible and executable in a sufficiently specified environment, and (iii) the documented workflow yields the same results as reported in the paper (within expected tolerances). This operationalization aligns with widely used definitions in computational statistics and scientific computing that emphasize "same data + same methods → same results," alongside transparent environments and documentation [20, 1, 3, 23]. We focus explicitly on statistical research software, so our assessment privileges artifacts typical of statistical analysis pipelines (data preprocessing, model fitting, simulation, visualization) and the practices that make them runnable and verifiable.

Despite growing awareness of reproducibility challenges in computational research, there is a lack of standardized methods to assess the reproducibility of research software artifacts. This study addresses the following research questions: how reproducibility can be operationalized and systematically evaluated in research papers involving software artifacts, what the essential criteria are that determine the reproducibility of computational research.

The objectives of this study are to establish a clear, operational definition of reproducibility within the scope of Statistical Research Software Engineering, to design a reproducibility assessment rubric tailored for evaluating research papers that include software artifacts, to apply this rubric to a selection of papers from the Journal of the American Statistical Association (JASA) and analyze reproducibility trends, and to provide insights and recommendations for improving reproducibility practices in research software development.

This study focuses on computational research papers published in English, specifically those whose code is made publicly available in the JASA online library across volumes from 2020 to 2025. The evaluation involves actual re-execution of the provided code to verify if it runs successfully and produces results consistent with those reported in the paper. The assessment is based solely on publicly available artifacts, including code, data, and documentation, as provided by the authors. Papers with missing or inaccessible artifacts were not excluded from the study. Instead, a score of zero was assigned to the corresponding indicators in the evaluation rubric. The study does not involve regenerating datasets, altering experimental setups, or validating results beyond the scope of the attached materials.

The remainder of this paper is organized as follows. Section 2 provides a comprehensive review of existing reproducibility frameworks and assessment methods in computational research, establishing the theoretical foundation for our approach. In Section 3, we present our methodology for measuring reproducibility in statistical software, including the development

of quantitative metrics and evaluation criteria. Section 4 describes the implementation of our assessment framework across a random sample of journal articles from JASA. The results of our reproducibility analysis are presented in Section 5, highlighting key patterns and identifying factors that contribute to higher or lower reproducibility scores. Section 6 discusses the implications of our findings for the statistical community and proposes best practices for improving reproducibility. Finally, Section 7 concludes with a summary of contributions and directions for future research.

2 Literature Review

The discourse on reproducibility is complicated by inconsistent terminology. Stodden et al. (2013) observed that terms like "reproducibility," "replicability," and "repeatability" are often used interchangeably, leading to confusion in cross-disciplinary communication. To address this, the Association for Computing Machinery (ACM) introduced its Artifact Review and Badging System (ACM, 2020), which distinguishes between repeatability, where the same team uses the same experimental setup, reproducibility, where different teams work with identical setups, and replicability, where different teams employ different experimental approaches. Barba (2018) further clarified these distinctions, proposing that reproducibility involves recreating results using the same input data, computational methods, and conditions, while replicability involves confirming findings through new data collection and similar methods. These foundational definitions are central to this study, as they help establish a shared language for evaluating reproducibility in computational research.

Across disciplines, reproducibility has been interpreted differently depending on methodological needs. In the life sciences, reproducibility emphasizes experimental protocols, reagent specifications, and accounting for biological variability (Errington et al., 2014). In physics, measurement precision, instrument calibration, and environmental control are paramount (Nosek & Errington, 2020). Social sciences focus more on transparency in participant sampling, survey design, and statistical analyses (Open Science Collaboration, 2015), while medicine emphasizes pre-registration of clinical trials, blinding procedures, and adherence to reporting standards such as CONSORT (Ioannidis, 2005). These variations illustrate that reproducibility is a multidimensional concept, shaped by the standards and constraints of individual fields.

In Statistics and Computer Science and Engineering, the concept of reproducibility takes on a computational form. Peng (2011) defines computational reproducibility as the ability to recreate analytical results given the same data and methods. This comes with unique challenges. Ivie and Thain (2018) describe the problem of "dependency hell," where rapidly evolving software ecosystems complicate execution. Hüttenhain et al. (2018) demonstrate how identical code may yield different outcomes on different hardware, particularly for parallel computing and machine learning applications. Deelman et al. (2018) emphasize difficulties when stochastic processes are involved, such as random initialization in machine learning models. Benureau and Rougier (2018) extend these ideas by identifying requirements for software-based research: code should be re-runnable (executes without errors), repeatable (produces consistent results across runs), reproducible (others can recreate results with documentation), reusable (repurposed for new applications), and replicable (val-

idated through independent implementations). Similarly, data reproducibility focuses on provenance, accessibility, and integrity, with the FAIR principles (Findable, Accessible, Interoperable, Reusable) gaining prominence as a standard framework (Wilkinson et al., 2016). Legal aspects, such as licensing, also play an important role, as noted by Stodden and Miguez (2014).

A growing body of technical solutions aims to address these challenges. Containerization technologies such as Docker package software with its dependencies to stabilize computational environments (Boettiger, 2015), while platforms like Code Ocean standardize reproducibility practices for scientific publishing (Clyburne-Sherin et al., 2019). Workflow management systems including Kepler, Taverna, and VisTrails capture provenance information and preserve parameter settings automatically (Freire et al., 2012). Literate programming tools such as Jupyter Notebooks, Quarto Documents and R Markdown weave together code, narrative, and results to promote transparency (Kluyver et al., 2016). Version control systems like Git, paired with platforms such as Gitlab and GitHub, further ensure traceability of code evolution and enable collaboration, while repositories like Zenodo, Figshare, and the Open Science Framework assign persistent identifiers to datasets and code, ensuring discoverability and citation (Stodden et al., 2016).

Beyond technical infrastructure, related studies provide insight into how reproducibility is conceptualized and practiced in different contexts. Chakravorti, Koneru, and Rajtmajer (2024) surveyed researchers in the USA and India, showing how cultural and resource constraints affect replicability and reproducibility. They found that while only 14.58% of Indian researchers and 33.96% of US researchers attempting replications reported affirmative results, attitudes toward open science practices are improving, albeit with concerns over time, resources, competition, and privacy. Desai, Abdelhamid, and Padalkar (2025) provide a hierarchical framework for validation studies, distinguishing between repeatability, dependent reproducibility, independent reproducibility, direct replicability, and conceptual replicability, thereby clarifying the layered ways in which findings can be verified. Feger and Woźniak (2022) argue for a researcher-centered definition, framing reproducibility around three components: ease of access to resources, completeness of metadata and documentation, and efficient interaction that minimizes wasted effort in re-executing experiments. Dreber and Johannesson (2025) extend this discussion to economics, proposing typologies such as robustness reproducibility and direct versus conceptual replicability, and introduce indicators like statistical significance, relative effect sizes, and variation across robustness tests to evaluate reproducibility and replicability systematically.

Taken together, these works demonstrate that statistical reproducibility is a complex and multi-layered concept. Foundational definitions provide clarity, disciplinary perspectives highlight variability in interpretation, technical solutions offer practical pathways, and field-specific frameworks propose systematic approaches to evaluation. Building on this literature, our study contributes a structured rubric that integrates these insights and applies them concretely to computational research papers in statistics, with the aim of advancing reproducibility practices within Statistical Research Software Engineering.

3 Methodology

This study adopts an observational research design to assess the reproducibility of computational research papers. The approach involves evaluating existing research artifacts such as datasets, code, and documentation, without intervening in the research process itself. The primary goal is to objectively observe and assess the reproducibility aspects as presented by the authors through a standardized assessment framework.

The scope of this study is limited to English-language computational research papers in statistics. Papers were selected from the Journal of the American Statistical Association (JASA), specifically from volumes 115 to 120, corresponding to the years 2020 to 2025. JASA was chosen for its strong emphasis on computational statistics and its structured policies on supplemental material availability, making it an ideal candidate for assessing reproducibility practices in research software engineering.

To ensure an unbiased selection, all papers from these volumes were listed chronologically in a spreadsheet and assigned unique serial numbers. A set of 20 papers was then selected using a true random number generator from `https://www.random.org/`, which uses atmospheric noise to generate randomness, as opposed to pseudo-random algorithms. Only papers with code artifacts available in the supplemental materials were considered eligible for the reproducibility assessment. The following table provides us with information on the papers selected using the aforementioned methodology:

S.No	Volume No.	Issue Year	Name
1	115	2020	Nonparametric Imputation by Data Depth
2	115	2020	GAP: A General Framework for Information Pooling in Two-Sample Sparse Inference
3	115	2020	Corrected Bayesian Information Criterion for Stochastic Block Models
4	115	2020	Adaptive Sparse Estimation With Side Information
5	116	2021	Efficiently Backtesting Conditional Value-at-Risk and Conditional Expected Shortfall
6	116	2021	The Impact of Churn on Client Value in Health Insurance, Evaluation Using a Random Forest Under Various Censoring Mechanisms
7	117	2022	Likelihood-Based Inference for Partially Observed Epidemics on Dynamic Networks
8	117	2022	Toward Causal Inference for Spatio-Temporal Data: Conflict and Forest Loss in Colombia
9	117	2022	Hierarchical Transformed Scale Mixtures for Flexible Modeling of Spatial Extremes on Datasets With Many Locations
10	118	2023	Conditional Separable Effects
11	118	2023	Semiparametric Goodness-of-Fit Test for Clustered Point Processes with a Shape-Constrained Pair Correlation Function
12	118	2023	Uniform Projection Designs and Strong Orthogonal Arrays
13	119	2024	A Semiparametric Inverse Reinforcement Learning Approach to Characterize Decision Making for Mental Disorders
14	119	2024	Fast and Numerically Stable Particle-Based Online Additive Smoothing: The AdaSmooth Algorithm
15	119	2024	Finite-dimensional Discrete Random Structures and Bayesian Clustering
16	119	2024	Variational Bayes for Fast and Accurate Empirical Likelihood Inference
17	119	2024	Variable Selection for High-Dimensional Nodal Attributes in Social Networks with Degree Heterogeneity
18	119	2024	Sparse Convoluted Rank Regression in High Dimensions
19	119	2024	Anytime-Valid Tests of Conditional Independence Under Model-X
20	120	2025	Modeling and Learning on High-Dimensional Matrix-Variate Sequences

Table 1: List of Selected Papers from JASA Volumes 115-120

3.1 Evaluation Framework Development

Based on insights from the literature review and a dry run of four papers featured as JASA reproducibility awardees, an evaluation framework was developed to systematically assess reproducibility in research software engineering. The framework identifies five key indicators that are critical for reproducibility: **Data Availability, Code Accessibility, Code**

Executability, Documentation Completeness, and **Result Reproducibility**. These indicators were selected to cover the entire lifecycle of reproducibility, from the initial presence of research artifacts to the final verification of published outcomes. Data and code availability are foundational: without them, reproducibility is impossible. Executability represents the practical ability to run the code and therefore carries more weight, as it directly determines whether the research workflow can be reproduced in practice. Documentation completeness was included because clear instructions, dependency specifications, and workflow descriptions often determine whether available code can actually be executed successfully. Finally, result reproducibility provides the highest level of assurance by validating whether the reproduced outputs align with those reported in the paper. Together, these five indicators capture both the presence and usability of artifacts as well as the ultimate success of reproducing results.

Indicator	Weight (%)
Data Availability	15
Code Accessibility	15
Code Executability	30
Documentation Completeness	20
Result Reproducibility	20

Table 2: Indicator weights used to compute the composite reproducibility score

The final normalized reproducibility score for each paper is computed using the formula:

$$\text{Final Score} = \frac{(0.15 \times DA) + (0.15 \times CA) + (0.30 \times CE) + (0.20 \times DC) + (0.20 \times RR)}{2}$$

where DA, CA, CE, DC, and RR represent the scores for Data Availability, Code Accessibility, Code Executability, Documentation Completeness, and Result Reproducibility respectively. This formula ensures that the final score ranges from 0 to 1.

To enhance the objectivity of the assessment, each indicator is evaluated with specific criteria for scores of 0, 1, and 2. This provides greater clarity on how judgments are made and ensures consistency across papers. For brevity, we provide only the high-level rationale above; the fine-grained 0/1/2 scoring criteria for each indicator are summarized in Table 3 and should be consulted when interpreting individual paper scores. The relative importance of indicators used in the composite score is given in Table 2.

Indicator	Score	Description
Data Availability	0	No data provided, or data is entirely inaccessible (e.g., proprietary without access, missing from supplementary materials).
	1	Partial data availability, such as data provided but requiring special permissions.
	2	Complete datasets provided, openly accessible, and clearly linked from the paper or supplementary materials.
Code Accessibility	0	No code provided or access restricted.
	1	Code is provided but incomplete (e.g., missing functions/scripts or references to unavailable repositories) or hosted in unstable locations.
	2	Full codebase provided, well-organized, openly accessible, and clearly linked from the paper.
Code Executability	0	Code fails to run due to missing dependencies, compilation errors, or other fatal errors, or incompatible environments, with no guidance to resolve issues.
	1	Code partially executes but requires manual fixes, undocumented dependencies, or non-trivial effort to resolve errors; some outputs may not match the original results.
	2	Code executes successfully in the stated environment with minimal setup, producing outputs consistent with the expected workflow.
Documentation Completeness	0	No documentation or instructions provided.
	1	Limited documentation, such as a brief README without details on environment setup, execution order, or dependency versions.
	2	Comprehensive documentation provided, including environment specifications (e.g., `requirements.txt`, Dockerfile, or `sessionInfo`), execution steps, and explanations of inputs/outputs.
Result Reproducibility	0	Results cannot be reproduced at all; outputs differ substantially or are missing (generally linked to code being unexecutable).
	1	Partial reproducibility achieved; some figures, tables, or analyses can be reproduced, but discrepancies remain compared to published results (generally linked to code being partially executable).
	2	Results are fully reproducible, with reproduced outputs matching those reported in the paper (within expected tolerances).

Table 3: Reproducibility scoring rubric used in this study

4 Implementation

Each selected paper underwent a structured evaluation workflow. The process began with downloading the supplementary materials associated with the paper, including datasets, source code, and documentation. The evaluation then proceeded through verification of data availability and accessibility, inspection of code accessibility and completeness, attempted execution of the provided code on a local machine or specified environment, assessment of documentation to determine if it provided sufficient instructions for reproduction, and comparison of reproduced results with the published outcomes to evaluate result reproducibility. For each step, observations were documented, and scores were assigned based on the reproducibility assessment rubric defined in tables 2 and 3

All programs were run on an Apple MacBook Air (M3, 8-core CPU, 10-core GPU, 16GB unified memory, 500GB SSD storage) running macOS Sequoia, with R version 4.5.0 and Python version 3.13.5. The reproducibility assessment primarily utilized R and RStudio for executing R scripts and R Markdown files, Python (Python3/VSCode) for papers that provided Python code and required environment and dependency management, and manual dependency installation for cases where environment specifications were missing and dependencies had to be resolved based on script errors. In several cases, issues such as missing dependencies, lack of environment specification, or compilation errors were documented as part of the evaluation.

5 Results and Analysis

The dataset for this study comprised 20 computational research papers selected randomly from JASA volumes 115–120. To ensure sufficient analytic depth while remaining resource-conscious, we set an a priori target of at least ten papers with nonzero reproducibility scores and sampled sequentially until that target was met; this process yielded 20 randomly drawn papers, of which 16 were evaluated and 10 achieved nonzero scores. Of those 16 papers: 6 lacked any code or data, several provided only partial artifacts, and 10 achieved non-zero reproducibility scores when assessed on five indicators—Data Availability, Code Accessibility, Code Executability, Documentation Completeness, and Result Reproducibility. To avoid repetition, we first summarize the papers that received a final score of 0 and then detail the papers with non-zero scores, placing all citations inline.

Papers with final score = 0

- *GAP: A General Framework for Information Pooling in Two-Sample Sparse Inference* (Xia, Cai, & Sun, 2020). Proposes a unifying strategy for pooling information across related two-sample sparse problems to improve estimation and testing in high dimensions. No supplemental materials were available; all indicators scored 0.

- *Corrected Bayesian Information Criterion for Stochastic Block Models* (Hu, Qin, Yan, & Zhao, 2020). Develops an adjusted BIC tailored to stochastic block models for more reliable selection of the number of communities. No artifacts were provided; all indicators scored 0.

- *Adaptive Sparse Estimation With Side Information* (Banerjee, Mukherjee, & Sun, 2020). Investigates using external/side information to improve accuracy in high-dimensional sparse estimation. No artifacts were provided; all indicators scored 0.

- *Efficiently Backtesting Conditional Value-at-Risk and Conditional Expected Shortfall* (Su, Qin, Peng, & Qin, 2021). Introduces procedures for backtesting CVaR and CES with attention to power and size properties in financial return series. No artifacts were provided; all indicators scored 0.

- *Conditional Separable Effects* (Stensrud, Robins, Sarvet, Tchetgen Tchetgen, & Young, 2023). Defines causal estimands that decompose treatment effects into separable components under stated identification conditions. No artifacts were provided; all indicators scored 0.

- *Uniform Projection Designs and Strong Orthogonal Arrays* (Sun & Tang, 2023). Constructs experimental designs with strong projection properties to support efficient exploration of high-dimensional input spaces. No artifacts were provided; all indicators scored 0.

Papers with non-zero final scores

- *Nonparametric Imputation by Data Depth* (Mozharovskyi, Josse, & Husson, 2020). Addresses missing-data imputation via depth-based methods designed to preserve multivariate structure. We executed the provided R code (RStudio, R 4.5.0) and verified that datasets and code were present (Data Availability = 2; Code Accessibility = 2), but compilation failures in the `imputeDepth` package prevented successful runs (Code Executability = 0); documentation was partial (1), and results could not be verified (0), yielding a final score of 0.4.

- *The Impact of Churn on Client Value in Health Insurance* (Gerber, Le Faou, Lopez, & Trupin, 2021). Quantifies how customer churn affects lifetime client value using actuarial modeling and predictive components. Data and code were available and ran under R 4.5.0 (scores of 2 for availability, accessibility, and executability); documentation lacked clear graph-reproduction steps and execution ordering (1), and reproducibility was partial (1)—for instance, the `sword` vignettes (`22980413_sword_vignette.Rmd`, `22980410_sword_vignette.Rmd`) executed in about three seconds each after installing `sword` via `devtools::install_github("YohannLeFaou/sword")`. Final score: 0.8.

- *Likelihood-Based Inference for Partially Observed Epidemics on Dynamic Networks* (Bu, Aiello, Xu, & Volfovsky, 2022). Develops likelihood-based tools for epidemic processes where both disease dynamics and network evolution are only partially observed. With R 4.5.0, data availability was limited to examples (1) though code was accessible (2); executability and documentation were partial (1, 1), and only partial reproduction was achieved (1): two Rmd cells ran in ∼12m30s total before a later cell failed with `object "recovers" not found`, preventing full verification. Final score: 0.575.

- *Toward Causal Inference for Spatio-Temporal Data: Conflict and Forest Loss in Colombia* (Christiansen, Baumann, Kuemmerle, Mahecha, & Peters, 2022). Presents a causal framework for spatio-temporal settings and applies it to forest loss and conflict data. Dataset generation used Python 3.13.5 and analysis used R 4.5.0; data and code were accessible (2, 2), executability was mixed with some scripts failing (e.g., missing `gdal` and an undocumented `bt` package), documentation was comprehensive (2), and reproducibility was partial (1). Reproducing Figure 3 required ∼74 minutes elapsed; several scripts ranged from sub-second to multi-hour runs with some failures (e.g., `resampling-sim.R` and `wrapper-results.R` halted on `hist.default` errors). Final score: 0.75.

- *Hierarchical Transformed Scale Mixtures for Flexible Modeling of Spatial Extremes* (Zhang, Shaby, & Wadsworth, 2022). Proposes a hierarchical class of transformed scale-mixture models to capture heavy tails and spatial extremal dependence across many sites. Although synthetic data and code were provided (2, 2), executability was only partial (1) due to unclear setup steps; documentation was insufficient (0), and results were unreproducible (0). On our macOS Sequoia/M3 system (R 4.5.0), compilation via `Rcpp` failed with an unresolved GSL symbol (`_gsl_integration_qagil`), blocking all analyses. Final score: 0.45.

- *Semiparametric Goodness-of-Fit Test for Clustered Point Processes* (Xu, Liang, Waagepetersen, & Guan, 2023). Develops semiparametric diagnostics for spatial/temporal point processes with clustering using a shape-constrained pair correlation function. Using R 4.5.0, data and code were available (2, 2) but executability was partial (1) due to missing CSVs and dependencies (e.g., `RandomFields`); documentation was absent (0) and reproducibility partial (1). Several scripts required minor edits (e.g., setting `$window <- as.polygonal(win)`); some tables and figures failed due to absent outputs, while others ran successfully within seconds to minutes. Final score: 0.55.

- *A Semiparametric Inverse Reinforcement Learning Approach* (Guo, Zeng, & Wang, 2024). Introduces an IRL method with semiparametric structure to infer reward functions or policies under limited assumptions. Data and code were present (2, 2); documentation and reproducibility were partial (1, 1), and full-scale runs required SLURM resources not available in our environment, though a local script (`run_local.R`) executed successfully with elapsed time ≈ 888s. Final score: 0.8.

- *Fast and Numerically Stable Particle-Based Online Additive Smoothing: The AdaSmooth Algorithm* (Mastrototaro, Olsson, & Alenlov, 2024). Proposes a particle-based online smoother emphasizing numerical stability and speed for state-space models. Implemented in Python 3.13.5 with standard dependencies (`numpy`, `scipy`, `tqdm`, `matplotlib`), the artifacts were complete and fully executable (2, 2, 2); documentation was clear (2), and results reproduced (2). Example runtimes were ∼17.4s for `testLGSS.py`, ∼2.1s for `testSamplingTimes.py`, and ∼10.2s for `testStocVol.py`. Final score: 0.9.

- *Finite-dimensional Discrete Random Structures and Bayesian Clustering* (Lijoi, Prunster, & Rigon, 2024). Studies Bayesian clustering and finite mixture structures with emphasis on discrete nonparametric priors and their induced partitions. Data and code were available and ran in R 4.5.0 (2, 2, 2); documentation was somewhat generic (1), but we reproduced the main results (2), with the primary driver (`MASTER_FILE_SIMULATION_STUDY.R`) completing in ∼1592s elapsed. Final score: 0.9.

- *Variational Bayes for Fast and Accurate Empirical Likelihood Inference* (Yu & Bondell, 2024). Presents a VB framework that accelerates empirical likelihood while aiming to preserve accuracy across common models. Although data and code were provided (2, 2) and documentation was adequate (2), executability failed in practice due to very long runtimes and missing objects in some notebooks, preventing reproduction (0, 0): for example, `LinearRegCondensed.Rmd` ran for ∼20 hours before crashing, and other materials failed with missing-object errors (e.g., `ARCHchan`, `xi`). Final score: 0.5.

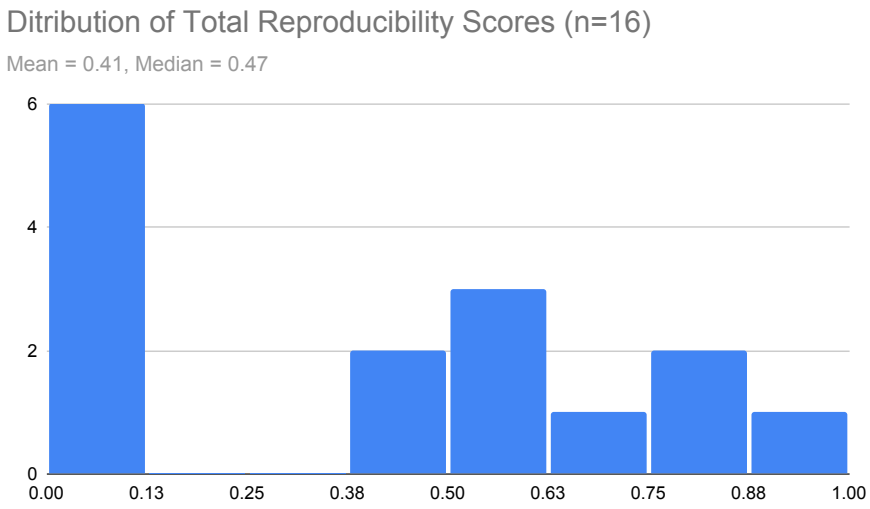

Figure 1: Distribution of total reproducibility scores across the 16 evaluated papers

Figure 1 displays the distribution of total reproducibility scores for the 16 evaluated papers. The scores are concentrated at 0 for six papers (37.5%), with the remaining ten forming a secondary mass between roughly 0.45 and 0.90; the mean score is 0.41 (SD = 0.36), the median is 0.48. This compact view highlights the split between non-executable or unreproducible artifacts and a smaller subset with moderate-to-high scores, providing a concise quantitative complement to the qualitative findings presented later.

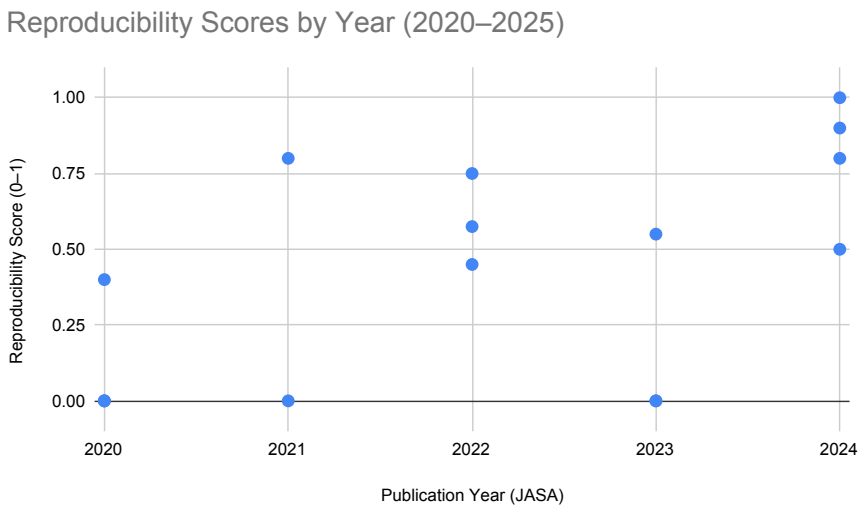

Figure 2: Reproducibility scores by year across the 16 evaluated papers

Figure 2 plots reproducibility scores against publication year to examine whether practices improved between 2020 and 2025. The plot shows that zero scores occurred in every year from 2020 through 2023 (with the exception of 2022), but 2024 stands out: three papers achieved the highest scores in our sample (0.8–1.0), while one paper scored 0.5, the lowest among the 2024 set . While the small sample size cautions against strong conclusions, the 2024 papers suggest a modest upward shift, with more articles achieving moderate-to-high reproducibility relative to earlier volumes. This pattern may indicate gradual improvements in artifact packaging and documentation practices in recent JASA publications.

6 Discussion

The reproducibility assessment revealed a significant disparity between artifact availability and actual reproducibility outcomes. Of the 16 evaluable papers, 9 had complete Data Availability (DA=2) and 10 had complete Code Accessibility (CA=2), indicating that datasets and code links were often present. However, execution and verification were much harder: among the 10 papers with nonzero overall scores, only 3 achieved full Code Executability (CE=2), only 3 had fully complete documentation (DC=2), and only 2 reproduced results in full (RR=2). Out of the 20 papers selected, only 10 achieved a nonzero reproducibility score, and only 6 demonstrated moderate to high reproducibility (final score ¿ 0.5). In effect, half of the assessed papers had artifacts that could not be used to reproduce results in any meaningful way, and fewer than one-third met even a moderate threshold of reproducibility.

The most prominent bottlenecks observed were in code executability, documentation completeness, and result reproducibility. 2 out of the 10 papers with non-zero scores provided code that was either not executable due to missing dependencies or required extensive manual intervention to resolve errors. 3 of them lacked any sort of documentation regarding environment setup, dependency versions, or execution steps, which made it difficult to replicate the authors' workflows even when artifacts were available. Even when code executed

successfully, replicating exact results was inconsistent due to missing runtime specifications, stochastic elements, or untracked preprocessing steps.

These findings highlight that mere availability of data and code is insufficient to guarantee reproducibility. Instead, several systemic factors noted in prior work likely contribute: (i) *incentive misalignment*, where novelty is rewarded more than artifact quality and maintenance [21, 45]; (ii) *environmental fragility* and dependency issues ("dependency hell") that make nominally available code hard to execute across machines [16, 23]; (iii) *heterogeneous hardware/software effects*, where identical code may yield divergent behavior or performance [14]; and (iv) *workflow complexity and long runtimes* that strain practical re-execution without robust packaging and provenance capture [8]. Prior studies emphasize containers and explicit environment specifications as mitigations [5, 23], alongside policies and incentives that recognize high-quality, reusable software and data [21, 45].

When compared to similar studies in other domains, our results align with broader concerns. Surveys in life sciences and social sciences have reported replication success rates of roughly 30–40% [11, 19], and economics reproducibility studies typically find computational reproducibility rates below 50% [10]. Our finding that only 30% of papers achieved moderate reproducibility is consistent with these trends, suggesting that computational statistics faces challenges of similar magnitude despite ostensibly greater transparency through code sharing.

Overall, our results underscore the need for *incentives and infrastructure* that make reproducibility routine rather than aspirational. In practice, this means (a) lightweight credit and recognition for executable, well-documented artifacts (e.g., badges or explicit citation credit), (b) first-class support for environment specification (e.g., Dockerfiles, Conda `environment.yml`, session info), and (c) minimal, journal-aligned checklists focused on executability and documentation quality. Prior work suggests pairing baseline expectations with positive "carrots" rather than relying solely on compliance "sticks" [45], and stresses transparency about environments and workflows as a cornerstone for computational reproducibility [23, 5].

In *statistics*, our assessment found that artifacts were often present (DA and CA frequently complete), yet reproducibility hinged on *executability*, *environment specification*, and *documentation*. Accordingly, we recommend that authors (i) package environments via containers or environment manifests, (ii) provide step-by-step run instructions and expected outputs, and (iii) include at least one validated reproduction run (ideally by someone other than the main developer). These recommendations align with established guidance that defines computational reproducibility as "same data + same methods $\rightarrow$ same results," contingent on transparent environments and documentation [20, 23, 3]. They are also consistent with community mechanisms that signal artifact quality (e.g., badges), which can function as incentives when coupled with clear criteria emphasizing *executability* rather than mere presence [1, 45]. More broadly, cross-disciplinary studies report modest replication/reproducibility success—e.g., psychology and social sciences [19], biomedicine [11], and economics [10]—which coheres with our statistical case study: availability is necessary but insufficient without runnable code, specified environments, and complete documentation. Containerization, workflow capture, and provenance have repeatedly been identified as practical enablers in such settings [5, 8, 16].

It is important, however, to acknowledge several limitations of this study. The assessment

was limited to 20 papers, with reproducibility experiments conducted on 16 of them, meaning a larger sample size would be required to generalize findings across the broader computational research domain. The evaluations also involved manual installation of dependencies and resolution of execution errors, which introduces subjective judgment that could vary across reviewers. Another limitation stems from the execution environment: all programs were run on an Apple MacBook Air (M3, 8-core CPU, 10-core GPU, 16GB unified memory, 500GB SSD storage) running macOS Sequoia, with R version 4.5.0 and Python version 3.13.5. Papers requiring specialized hardware, such as SLURM clusters or GPUs, were only partially assessed, reducing the completeness of executability evaluation. Finally, although assessments were cross-reviewed, no formal inter-rater reliability metrics were computed, which may affect the objectivity of scoring.

Of the 20 randomly selected JASA papers, four did not meet the inclusion criterion of providing statistical research software artifacts suitable for assessment (no executable code, purely theoretical contributions without computational materials, or irretrievably broken links). As stated in Section 5, we report detailed results for the 16 evaluable papers; we also set an a priori target of obtaining at least ten nonzero scores to support indicator-level comparisons, after which we did not expand the evaluation further given compute/time constraints.

7 Conclusion and Future Work

This study presented a systematic reproducibility assessment of computational research papers from the Journal of the American Statistical Association (JASA). A structured evaluation rubric was proposed, encompassing five key indicators: Data Availability, Code Accessibility, Code Executability, Documentation Completeness, and Result Reproducibility. The assessment revealed that while artifact availability (data and code) is becoming common, actual reproducibility of results remains inconsistent due to insufficient documentation, lack of environment specifications, and code executability issues. Taken together, the study offers a practical framework to objectively assess reproducibility in research software engineering.

Building on these findings, we recommend several practices to improve reproducibility in computational research. Authors should provide executable artifacts using environment packaging tools like Docker or Conda, and comprehensive documentation outlining the setup, execution steps, and dependency versions must accompany code artifacts. Journals should pair clear baseline expectations with positive incentives ("carrots"); for example, badges or recognition explicitly tied to *executability* and independently validated reproducibility, formal credit for reusable software and datasets, and citation practices that reward high-quality artifacts—rather than relying solely on compliance "sticks" [45]. Furthermore, a reproducibility badge system that is beyond mere availability, one that accounts for artifact executability and result validation, should be adopted.

Looking ahead, there are several avenues for future research that could extend and deepen this work. Expanding the assessment to a larger corpus of papers across multiple journals and disciplines would help validate the generalizability of the rubric. At the same time, developing automated tools for reproducibility scoring could streamline artifact validation and reduce manual evaluation overhead. Another promising direction lies in investigating

the impact of reproducibility badges and open science incentives on the quality of research software artifacts. Finally, longitudinal studies could provide valuable insights into how reproducibility practices evolve over time as awareness grows and standards continue to mature.

References

[1] ACM. (2020). Artifact Review and Badging Version 2.0. Association for Computing Machinery. https://cacm.acm.org/opinion/investigating-research-software-engineering-toward-rse-research/

[2] Baker, M. (2016). 1,500 scientists lift the lid on reproducibility. *Nature*, 533(7604), 452–454. https://doi.org/10.1038/533452a

[3] Barba, L. A. (2018). Terminologies for reproducible research [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1802.03311

[4] Benureau, F. C., & Rougier, N. P. (2018). Re-run, repeat, reproduce, reuse, replicate: Transforming code into scientific contributions. *Frontiers in Neuroinformatics*, 11, 69. https://doi.org/10.3389/fninf.2017.00069

[5] Boettiger, C. (2015). An introduction to Docker for reproducible research. *ACM SIGOPS Operating Systems Review*, 49(1), 71–79. https://doi.org/10.1145/2723872.2723882

[6] Chakravorti, T., Koneru, S. D., & Rajtmajer, S. (2024). Reproducibility, replicability, and transparency in research: What 430 professors think in universities across the USA and India [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2402.08796

[7] Clyburne-Sherin, A., Fei, X., & Green, S. A. (2019). Computational reproducibility via containers in psychology. *Meta-Psychology*, 3. https://doi.org/10.15626/MP.2019.15022163

[8] Deelman, E., Peterka, T., Altintas, I., Carothers, C. D., van Dam, K. K., Moreland, K., ... & Vetter, J. S. (2018). The future of scientific workflows. *The International Journal of High Performance Computing Applications*, 32(1), 159–175. https://doi.org/10.1177/1094342017719968

[9] Desai, A., Abdelhamid, M., & Padalkar, N. R. (2025). What is reproducibility in artificial intelligence and machine learning research? *AI Magazine*, 46(2), e70004. https://doi.org/10.1002/aaai.70004

[10] Dreber, A., & Johannesson, M. (2025). A framework for evaluating reproducibility and replicability in economics. *Economic Inquiry*, 63(2), 338–356. https://doi.org/10.1111/ecin.13244

[11] Errington, T. M., Iorns, E., Gunn, W., Tan, F. E., Lomax, J., & Nosek, B. A. (2014). Science forum: An open investigation of the reproducibility of cancer biology research. *eLife*, 3, e04333. https://doi.org/10.7554/eLife.04333

[12] Feger, S. S., & Wo'zniak, P. W. (2022). Reproducibility: A researcher-centered definition. *Multimodal Technologies and Interaction*, 6(2), 17. https://doi.org/10.3390/mti6020017

[13] Freire, J., Bonnet, P., & Shasha, D. (2012). Computational reproducibility: State-of-the-art, challenges, and database research opportunities. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data* (pp. 593–596). https://doi.org/10.1145/2213836.2213895

[14] Huttenhain, J., Cramer, T., Byna, S., Guti'errez, A., & Schulthess, T. C. (2018). On the correctness and performance of parallel programs for distributed-memory multiprocessors. *Parallel Computing*, 75, 51–63. https://doi.org/10.1016/j.parco.2018.05.006

[15] Ioannidis, J. P. A. (2005). Why most published research findings are false. *PLoS Medicine*, 2(8), e124. https://doi.org/10.1371/journal.pmed.0020124

[16] Ivie, P., & Thain, D. (2018). Reproducibility in scientific computing. *ACM Computing Surveys*, 51(3), 1–36. https://doi.org/10.1145/3191513

[17] Kluyver, T., Ragan-Kelley, B., P'erez, F., Granger, B. E., Bussonnier, M., Frederic, J., ... & Jupyter Development Team. (2016). Jupyter Notebooks—a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas* (pp. 87–90). IOS Press. https://doi.org/10.3233/978-1-61499-649-1-87

[18] Nosek, B. A., & Errington, T. M. (2020). What is replication? *PLoS Biology*, 18(3), e3000691. https://doi.org/10.1371/journal.pbio.3000691

[19] Open Science Collaboration. (2015). Estimating the reproducibility of psychological science. *Science*, 349(6251), aac4716. https://doi.org/10.1126/science.aac4716

[20] Peng, R. D. (2011). Reproducible research in computational science. *Science*, 334(6060), 1226–1227. https://doi.org/10.1126/science.1213847

[21] Stodden, V., Guo, P., & Ma, Z. (2013). Toward reproducible computational research: An empirical analysis of data and code policy adoption by journals. *PLOS ONE*, 8(6), e67111. https://doi.org/10.1371/journal.pone.0067111

[22] Stodden, V., & Miguez, S. (2014). Best practices for computational science: Software infrastructure and environments for reproducible and extensible research. *Journal of Open Research Software*, 2(1). https://doi.org/10.5334/jors.ay

[23] Stodden, V., McNutt, M., Bailey, D. H., Deelman, E., Gil, Y., Hanson, B., ... & Taufer, M. (2016). Enhancing reproducibility for computational methods. *Science*, 354(6317), 1240–1241. https://doi.org/10.1126/science.aah6168

[24] Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., ... & Mons, B. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3, 160018. https://doi.org/10.1038/sdata.2016.18

[25] Mozharovskyi, P., Josse, J., & Husson, F. (2020). Nonparametric imputation by data depth. *Journal of the American Statistical Association*, 115(529), 241–253. https://doi.org/10.1080/01621459.2018.1543123

[26] Xia, Y., Cai, T. T., & Sun, W. (2020). GAP: A general framework for information pooling in two-sample sparse inference. *Journal of the American Statistical Association*. https://doi.org/10.1080/01621459.2019.1611585

[27] Hu, J., Qin, H., Yan, T., & Zhao, Y. (2020). Corrected Bayesian information criterion for stochastic block models. *Journal of the American Statistical Association*, 115(532), 1771–1783. https://doi.org/10.1080/01621459.2019.1637744

[28] Banerjee, T., Mukherjee, G., & Sun, W. (2020). Adaptive sparse estimation with side information. *Journal of the American Statistical Association*, 115(532), 2053–2067. https://doi.org/10.1080/01621459.2019.1679639

[29] Su, Q., Qin, Z., Peng, L., & Qin, G. (2021). Efficiently backtesting conditional value-at-risk and conditional expected shortfall. *Journal of the American Statistical Association*, 116(536), 2041–2052. https://doi.org/10.1080/01621459.2020.1763804

[30] Gerber, G., Faou, Y. L., Lopez, O., & Trupin, M. (2021). The impact of churn on client value in health insurance, evaluation using a random forest under various censoring mechanisms. *Journal of the American Statistical Association*, 116(536), 2053–2064. https://doi.org/10.1080/01621459.2020.1764364

[31] Bu, F., Aiello, A. E., Xu, J., & Volfovsky, A. (2022). Likelihood-based inference for partially observed epidemics on dynamic networks. *Journal of the American Statistical Association*, 117(537), 510–526. https://doi.org/10.1080/01621459.2020.1790376

[32] Christiansen, R., Baumann, M., Kuemmerle, T., Mahecha, M. D., & Peters, J. (2022). Toward causal inference for spatio-temporal data: Conflict and forest loss in Colombia. *Journal of the American Statistical Association*, 117(538), 591–601. https://doi.org/10.1080/01621459.2021.2013241

[33] Zhang, L., Shaby, B. A., & Wadsworth, J. L. (2022). Hierarchical transformed scale mixtures for flexible modeling of spatial extremes on datasets with many locations. *Journal of the American Statistical Association*, 117(539), 1357–1369. https://doi.org/10.1080/01621459.2020.1858838

[34] Stensrud, M. J., Robins, J. M., Sarvet, A., Tchetgen Tchetgen, E. J., & Young, J. G. (2023). Conditional separable effects. *Journal of the American Statistical Association*, 118(544), 2671–2683. https://doi.org/10.1080/01621459.2022.2071276

[35] Xu, G., Liang, C., Waagepetersen, R., & Guan, Y. (2023). Semiparametric goodness-of-fit test for clustered point processes with a shape-constrained pair correlation function. *Journal of the American Statistical Association*, 118(543), 2072–2087. https://doi.org/10.1080/01621459.2022.2029456

[36] Sun, C. Y., & Tang, B. (2023). Uniform projection designs and strong orthogonal arrays. *Journal of the American Statistical Association*, 118(541), 417–423. https://doi.org/10.1080/01621459.2021.1935268

[37] Guo, X., Zeng, D., & Wang, Y. (2024). A semiparametric inverse reinforcement learning approach to characterize decision making for mental disorders. *Journal of the American Statistical Association*, 119(545), 27–38. https://doi.org/10.1080/01621459.2023.2261184

[38] Mastrototaro, A., Olsson, J., & Alenlov, J. (2024). Fast and numerically stable particle-based online additive smoothing: the adasmooth algorithm. *Journal of the American Statistical Association*, 119(545), 356–367. https://doi.org/10.1080/01621459.2022.2118602

[39] Lijoi, A., Prunster, I., & Rigon, T. (2024). Finite-dimensional discrete random structures and Bayesian clustering. *Journal of the American Statistical Association*, 119(546), 929–941. https://doi.org/10.1080/01621459.2022.2149406

[40] Yu, W., & Bondell, H. D. (2024). Variational Bayes for fast and accurate empirical likelihood inference. *Journal of the American Statistical Association*, 119(546), 1089–1101. https://doi.org/10.1080/01621459.2023.2169701

[41] Wang, J., Cai, X., Niu, X., & Li, R. (2024). Variable selection for high-dimensional nodal attributes in social networks with degree heterogeneity. *Journal of the American Statistical Association*, 119(546), 1322–1335. https://doi.org/10.1080/01621459.2023.2187815

[42] Zhou, L., Wang, B., & Zou, H. (2024). Sparse convoluted rank regression in high dimensions. *Journal of the American Statistical Association*, 119(546), 1500–1512. https://doi.org/10.1080/01621459.2023.2202433

[43] Grunwald, P., Henzi, A., & Lardy, T. (2024). Anytime-valid tests of conditional independence under model-X. *Journal of the American Statistical Association*, 119(546), 1554–1565. https://doi.org/10.1080/01621459.2023.2205607

[44] Zhang, X., Liu, C. C., Guo, J., Yuen, K. C., & Welsh, A. H. (2025). Modeling and learning on high-dimensional matrix-variate sequences. *Journal of the American Statistical Association*, 120(549), 419–434. https://doi.org/10.1080/01621459.2024.2344687

[45] Leonelli, S., Spichtinger, D., & Prainsack, B. (2015). Sticks AND Carrots: Encouraging Open Science at its source. *Geo: Geography and Environment*, 2(1), 12–16. https://pmc.ncbi.nlm.nih.gov/articles/PMC4591465/